



INDUS11

AI Financial Risk & Fraud Decision Engine

FIRST PROJECT REVIEW · July 11, 2026

Joshi Om · Krish Gajera · Drashti Dedaniya

Semester Group Project · Algoma University

Real-time fraud decisions, explained in plain English

The problem

Banks must decide in milliseconds whether a payment is legitimate. Pure rule systems miss novel fraud; pure ML systems can't explain why they blocked a customer.

Our idea

- Score every transaction with three independent engines — business rules, graph analysis of fraud rings, and a RAG-grounded LLM
- Combine them into one 0-100 composite score with clear thresholds
- Return the decision plus a natural-language explanation an analyst can act on



APPROVE score 0-39

Clean payment — release instantly



REVIEW score 40-69

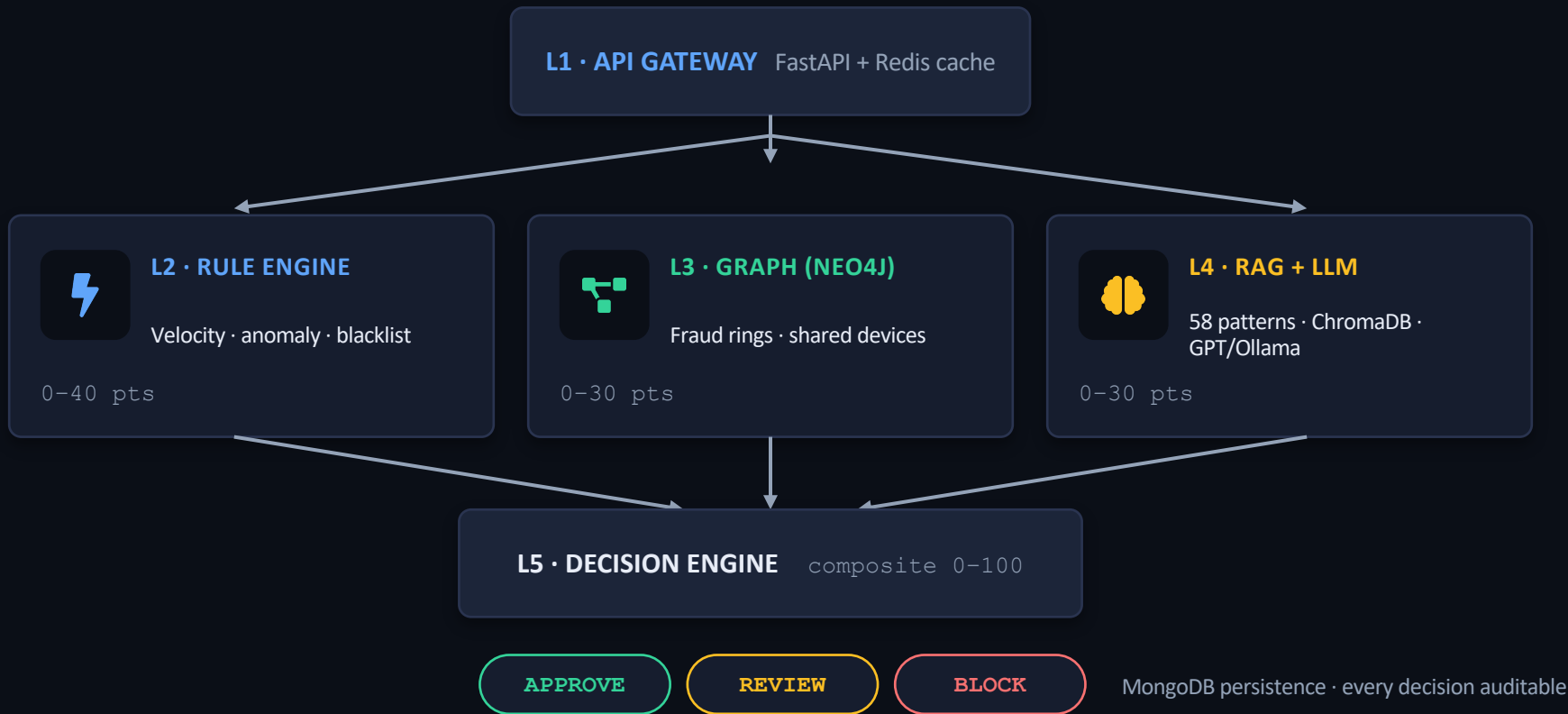
Suspicious — queue for an analyst



BLOCK score 70-100

High risk — stop the transaction

Five-layer pipeline — three engines run in parallel



Production-grade stack, fully containerised



FastAPI

Async Python 3.12 API — 12 REST endpoints



MongoDB 7

Transactions & accounts · Beanie ODM



Neo4j 5

Fraud-ring graph · Cypher pattern queries



Redis 7

Profile cache · sorted-set velocity windows



LangChain

RAG pipeline · OpenAI with Ollama fallback



ChromaDB

Vector store · 58 fraud-pattern documents



React + Vite

Live ops dashboard with Recharts



Docker Compose

One-command spin-up of all services

Technologies evaluated — and what we chose

Category	Our choice	Alternatives evaluated
API framework	FastAPI	Flask · Django REST · Node/Express
Document DB	MongoDB + Beanie	PostgreSQL · DynamoDB · CouchDB
Graph DB	Neo4j	Amazon Neptune · ArangoDB · TigerGraph
Cache	Redis	Memcached · in-process LRU
Vector store	ChromaDB	FAISS · Pinecone · Qdrant · pgvector
LLM / RAG	LangChain + OpenAI / Ollama	Anthropic Claude · Gemini · Mistral
Frontend	React + Vite + Recharts	Vue · Svelte · Chart.js
Containers / infra	Docker Compose	Kubernetes · Railway · Render

Choices favour open-source, containerised, and locally-runnable tools — so the whole stack spins up on one laptop with no paid services.

Task distribution — clear ownership per layer



Joshi Om

Member A

Platform & Rules

- FastAPI server & REST API design
- MongoDB schema · Beanie ODM
- Redis caching & velocity windows
- Rule Engine (Layer 2)
- Rate limiting · Docker Compose



Krish Gajera

Member B

Graph Intelligence

- Neo4j schema & Cypher fraud queries
- Graph Analyzer (Layer 3)
- Synthetic fraud dataset — 500 accounts
- Fraud-ring label propagation
- Graph exploration API



Drashti Dedaniya

Member C

AI & Experience

- RAG pipeline — LangChain + ChromaDB
- Prompt engineering & LLM fallback
- Decision Engine (Layer 5)
- React dashboard & stats API
- Fraud-pattern knowledge base (58 docs)

Planning & execution — four phases, three complete



DONE

Phase 1 · Design

Jun 9 – Jun 20

- Architecture & 5-layer design
- API contract (Pydantic schemas)
- Repo scaffold · Docker Compose



DONE

Phase 2 · Core build

Jun 23 – Jul 1

- All 5 layers implemented
- Rule, Graph, RAG, Decision engines
- 8 unit tests passing



DONE

Phase 3 · Data + UX

Jul 2 – Jul 4

- Seed scripts (MongoDB + Neo4j rings)
- 58-doc fraud knowledge base
- React dashboard + stats API



IN PROGRESS

Phase 4 · Integration

Jul 5 – Jul 10

- Full-stack run under Docker
- End-to-end demo rehearsal
- Ollama offline path verified

What is built and verified today

12

REST endpoints

transactions · accounts · graph · stats

8/8

unit tests passing

rule + decision engines, pytest

58

fraud patterns

RAG knowledge base in ChromaDB

500

synthetic accounts

3 mule rings · ~1,000 transactions



Verified

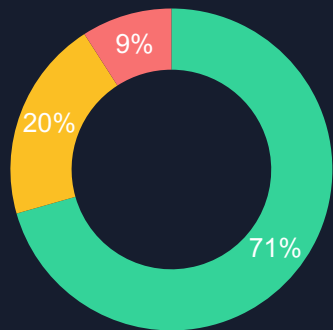
- pytest suite green (8/8) — rule engine & decision thresholds
- App boots with all routers registered; dashboard builds clean with zero console errors
- Deterministic seed datasets reproduce identical fraud graphs for every member
- Ollama fallback wired so the demo runs with no API keys and no internet

Live dashboard — decisions as they happen

Demo flow (3 minutes)

1. Submit a normal \$250 retail payment → APPROVE in <200 ms
2. Replay a \$9,450 wire from a high-risk account → BLOCK with mule-ring evidence from Neo4j
3. Burst 6 rapid payments → velocity rule flags REVIEW
4. Read the LLM explanation — the panel sees why, not just a score
5. Dashboard KPIs & charts update live (10 s refresh)

Sample run — decision mix



■ Approve ■ Review ■ Block

187 transactions · synthetic dataset

Remaining work — five weeks to final delivery

When	Focus	Owner	Deliverable
Jul 5–10	Full-stack integration under Docker; demo rehearsal	All	Working end-to-end demo for Review 1
Jul 14–24	Accuracy evaluation on synthetic set; tune thresholds & prompts	B + C	Precision/recall report; tuned scoring
Jul 14–24	Auth (API keys), request logging, error handling hardening	A	Secured, observable API
Jul 27–Aug 7	Graph visualisation in dashboard; load test 100 tx/s	B + C	Fraud-ring explorer view; perf numbers
Aug 10–14	Final report, documentation, presentation & recorded demo	All	Final submission package

Milestones: Review 1 — Jul 11 · Review 2 (expected) — early Aug · Final demo — Aug 14

What could go wrong — and our mitigations



LLM score drift

Same transaction, different scores across runs

Mitigation: Temperature 0, few-shot anchors, LLM capped at 30/100 — rules & graph dominate



API cost / availability

OpenAI quota or network failure during demo

Mitigation: Automatic fallback to local Ollama; dashboard has offline sample mode



Integration complexity

4 services (MongoDB, Neo4j, Redis, API) must run together

Mitigation: Docker Compose one-command startup; deterministic seed scripts; health endpoint

Thank you — questions welcome